Fila Dinâmica

Professor: André Felipe Ribeiro Cordeiro
Disciplina: Estruturas de Dados

Departamento de Informática
Centro de Tecnologia
Universidade Estadual de Maringá

Introdução

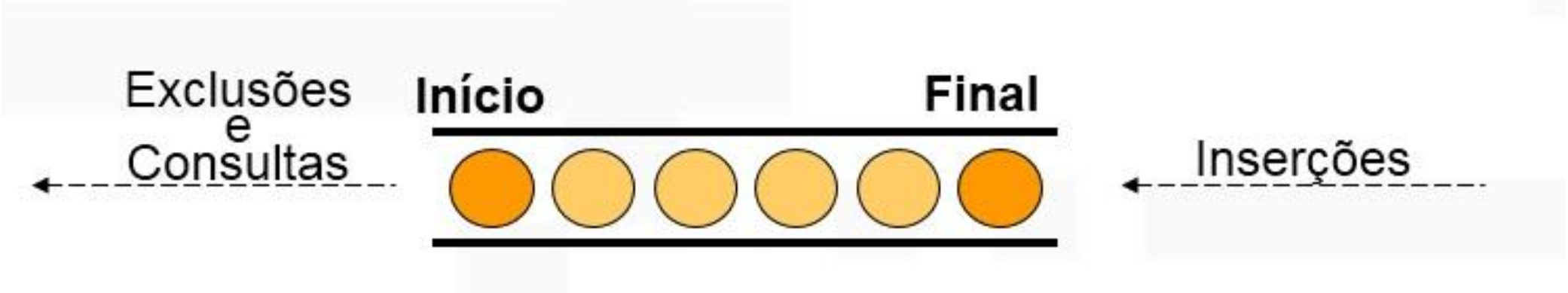
- Outra estrutura de dados bastante utilizada em computação é a **fila**.

- Na estrutura de fila, existem **regras** para o **acesso**, **inserção** e **remoção** de **itens**.

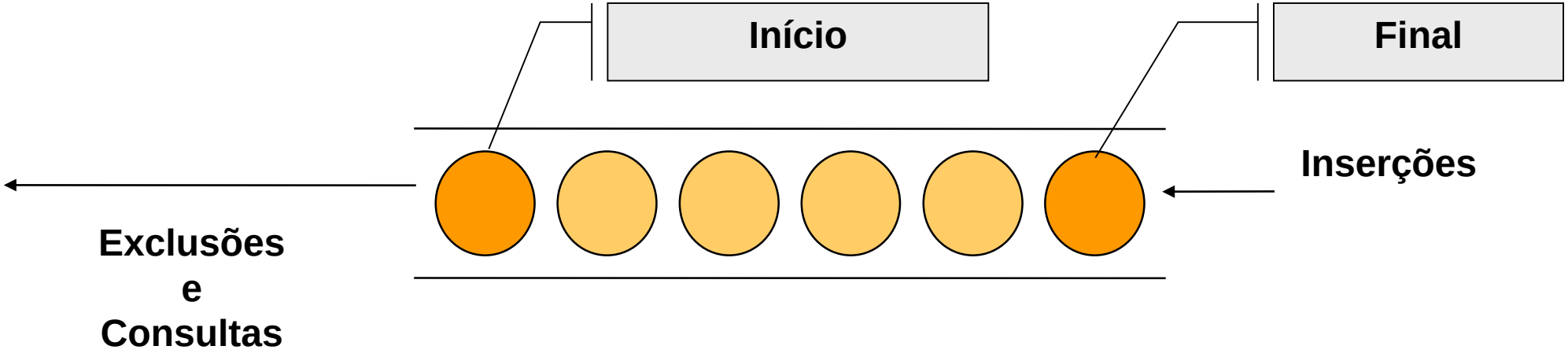
Regras

- **Acesso**
  - Dependente do TAD e/ou da aplicação.
- **Inserção**
  - Insere-se o item no **final da fila**.
- **Remoção**
  - Remove-se o item do **início da fila**.

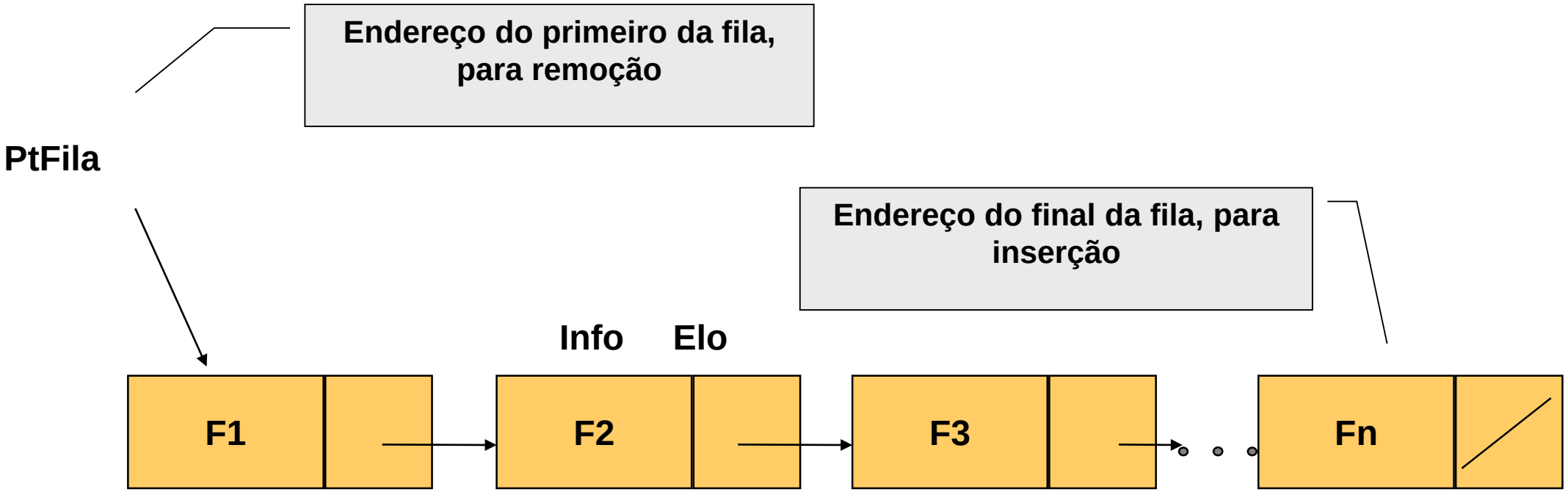
Representação



Representação



Representação



Para acessar o último nodo, é necessário percorrer toda a fila a partir do primeiro nodo

Observações

- As **estruturas** de **pilha** e **fila** se **diferenciam** pela **ordem de saída** dos itens.

- Cada estrutura é organizada de uma **maneira específica**.

Observações

- **Pilha**
  - O último item que entra é o primeiro que sai.
  - **Last In**, **First Out** (LIFO)

- **Fila**
  - O primeiro item que entra é o primeiro que sai.
  - **First In**, **First Out** (FIFO)

Operações
Fila

- As seguintes operações podem ser utilizadas em diferentes aplicações:
  - **inicializar** a fila
  - **verificar** se a fila está **vazia**
  - **enfileirar item** na fila
  - **desenfileirar item** da fila
  - **imprimir** os **elementos** da fila

Implementação

```
#include <stdlib.h>
#include <stdio.h>
```

```
typedef struct TipoCelula *TipoApontador;
typedef int TipoChave;

typedef struct TipoItem {
  TipoChave chave;
  /* outros componentes */
} TipoItem;
```

Implementação

```
typedef struct TipoCelula {
  TipoItem item;
  TipoApontador proximo;
} TipoCelula;
```

```
typedef struct TipoFila {
  TipoApontador frente, tras;
} TipoFila;
```

Implementação

```
void Inicializa(TipoFila *fila){
  fila->frente =
     (TipoApontador) malloc (sizeof(TipoCelula));
  fila->tras = fila->frente;
  fila->frente->proximo = NULL;
}
```

```
int Vazia(TipoFila fila){
  return (fila.frente == fila.tras);
}
```

Implementação

```
void Enfileira(TipoItem elemento,
                                TipoFila *fila){
  fila->tras->proximo =
     (TipoApontador) malloc (sizeof(TipoCelula));
  fila->tras = fila->tras->proximo;
  fila->tras->item = elemento;
  fila->tras->proximo = NULL;
}
```

Implementação

```
void Desenfileira(TipoFila *fila,
                                  TipoItem *elemento){
  TipoApontador p_auxiliar;
  if (Vazia(*fila)){
    printf("Erro fila esta vazia\n");
    return;
  }
  p_auxiliar = fila->frente;
  fila->frente = fila->frente->proximo;
  *elemento = fila->frente->item;
  free(p_auxiliar);
}
```

Implementação

```
void Imprime(TipoFila fila){
  TipoApontador ponteiro;
  ponteiro = fila.frente->proximo;

  while (ponteiro != NULL) {
    printf("%d\n", ponteiro->item.chave);
    ponteiro = ponteiro->proximo;
  }
}
```

Observações

- Outras operações poderiam ser consideradas, além das apresentadas.

- **Exemplos**
  - **Encontrar** o **menor elemento** da fila
  - **Verificar** se a fila possui **elementos repetidos**

Observações

- A estrutura de fila apresentada está organizada tal como outras filas “**não computacionais**”.

- Em um banco, a **primeira pessoa da fila** é a **primeira** a ser **atendida** (cenário esperado!).

Referências

- Introdução a Estruturas de Dados, Waldemar Celes, Renato Cerqueira e José Rangel, 2016.

- Projeto de Algoritmos com implementações em Pascal e C, Nivio Ziviani, 2011.

- Aulas Professor Rodolfo Miranda Pereira, UEM.